

Lecture 5: Robust Training

CS 580B1: Trustworthy Machine Learning

Fall 2024

Acknowledgments

Course slides derived from material created by Rajeev Alur and Osbert Bastani at UPenn ([link](#) to their course)

Slides from [Adversarial Robustness - Theory and Practice](#) tutorial at NeurIPS'18 by Zico Kolter and Aleksander Madry

Other relevant courses:

[CS 329T](#) at Stanford

[CS 521](#) at UIUC

[ECE1784H/CSC2559H](#) at U Toronto

Course logistics

- Reviews:
 - Review due tomorrow at 11:59 PM
 - Paper to review: [Certified Adversarial Robustness via Randomized Smoothing](#) by Cohen et al.
 - Mention names of those who you discussed the paper with in your review
- Presentations:
 - Check course schedule and presentation teams; small updates to both
 - Submit slides on Canvas after presentation
 - Teams presenting should come see me in office hours before the presentation
- Can submit anonymous feedback about course during the semester

Quick recap

Last week:

- Definition of adversarial examples and local robustness of neural networks
- Algorithms for finding adversarial examples, i.e., attack algorithms
 - FGSM
 - PGD
 - Carlini-Wagner attacks
- All attack algorithms based on solving *constrained optimization* problems via gradient descent

Approaches for defending against adversarial examples

- Test time
- Inference time
- Train time

Approaches for defending against adversarial examples

Test time:

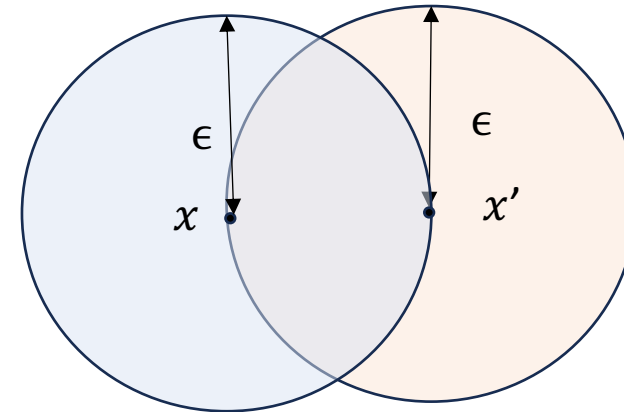
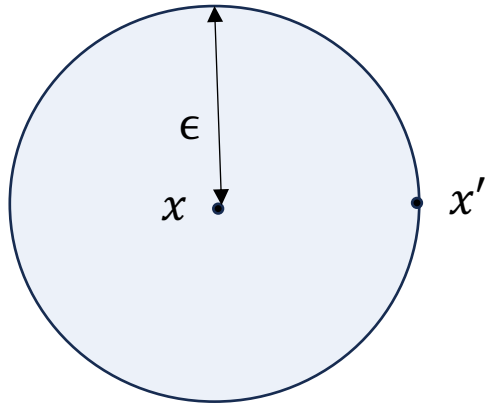
$$\text{Robust Accuracy} = \frac{\text{\# of test samples where model is accurate and locally robust}}{\text{\# of test samples}}$$

- Measure upper bound on *robust accuracy*
 - Use attack algorithms
 - If attack succeeds at x , model is NOT robust at x . Otherwise, don't know
- Measure lower bound on *robust accuracy*
 - Use robustness certification algorithms
 - If certification succeeds at x , model is robust at x . Otherwise, don't know

Approaches for defending against adversarial examples

Inference time:

- Try to attack or certify model at inference
- Reject input if attack succeeds or certification fails



Approaches for defending against adversarial examples

Train time:

- Modify training process to increase model robustness, i.e., make it less susceptible to adversarial attacks
- Modifications to
 - Optimization problem
 - Architecture
 - Data

Upcoming agenda

- (Last week) Attack algorithms for finding adversarial examples
- (This week) Defending against adversarial examples via training
 - [Towards Deep Learning Models Resistant to Adversarial Attacks](#) by Madry et al.
 - [Certified Adversarial Robustness via Randomized Smoothing](#) by Cohen et al.
- (Sept 10 and 12) Introduction to formal methods
- (Sept 17 and 19) Certifying local robustness of DNNs using formal methods

Finding adversarial examples: Optimization formulation

Given model f_θ and input x , the search for an adversarial example wrt perturbation set Δ can be framed as:

$$\operatorname{argmax}_{\delta \in \Delta} L(f(x + \delta), f(x))$$

Training robust models: Optimization formulation

Given model f_θ and training dataset of n pairs of (x_i, y_i) samples, the problem of training a robust classifier wrt perturbation set Δ can be framed as:

$$\operatorname{argmin}_{\theta} \sum_{i=1}^n \max_{\delta \in \Delta} L(f_\theta(x_i + \delta), y_i)$$

|
Finding adversarial example

Adversarial training

How do we solve the minmax optimization problem?

$$\operatorname{argmin}_{\theta} \sum_{i=1}^n \max_{\delta \in \Delta} L(f_{\theta}(x_i + \delta), y_i)$$

Would like to use gradient descent but how do we compute gradient wrt to the max term inside?

Danskin's theorem

A fundamental result in optimization:

$$\nabla_{\theta} \max_{\delta \in \Delta} L(f_{\theta}(x + \delta), y) = \nabla_{\theta} L(f_{\theta}(x + \delta^*), y)$$

where $\delta^* = \arg\max_{\delta \in \Delta} L(f_{\theta}(x + \delta), y)$

Seems “obvious,” but a subtle result; means we can optimize through the max by just finding it's maximizing value

Note: Theorem only applies when max is performed exactly

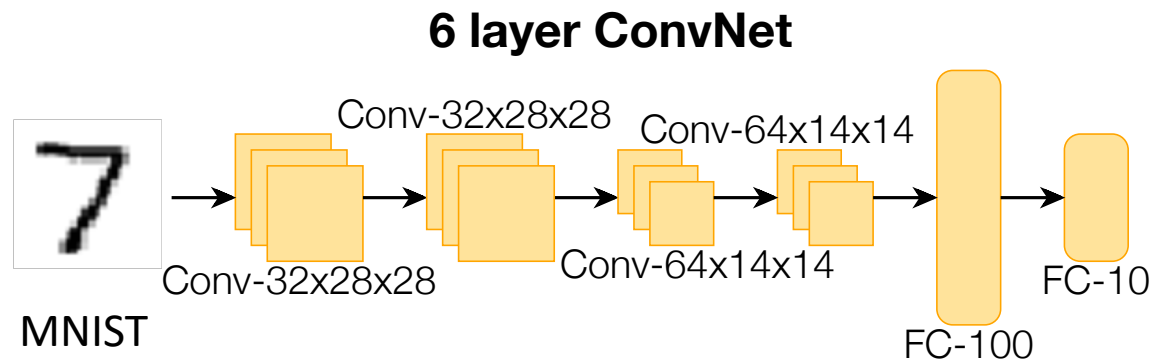
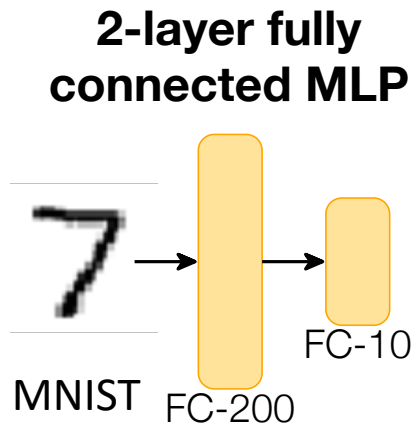
Adversarial training algorithm

```
 $\theta_0 \leftarrow \text{Initialize}()$   
 $t \leftarrow 0$   
Until convergence or budget exhaustion:  
  For each  $(x_i, y_i)$  in training data:  
     $\delta_i^* \leftarrow \underset{\delta \in \Delta}{\operatorname{argmax}} L(f_{\theta_t}(x_i + \delta), y_i)$   
     $\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{n} \nabla_{\theta} \left( \sum_{i=1}^n L(f_{\theta_t}(x_i + \delta_i^*), y_i) \right)$   
     $t \leftarrow t + 1$   
return  $f_{\theta_t}$ 
```

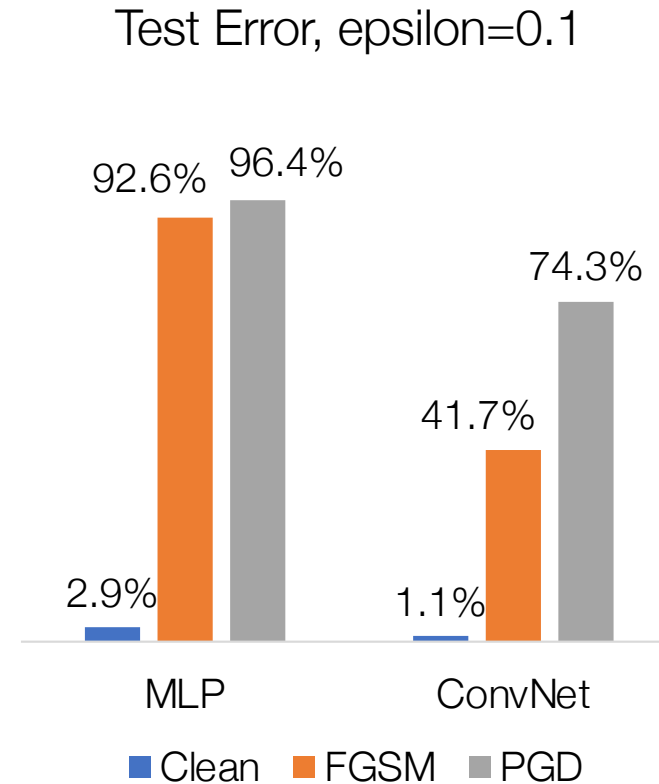
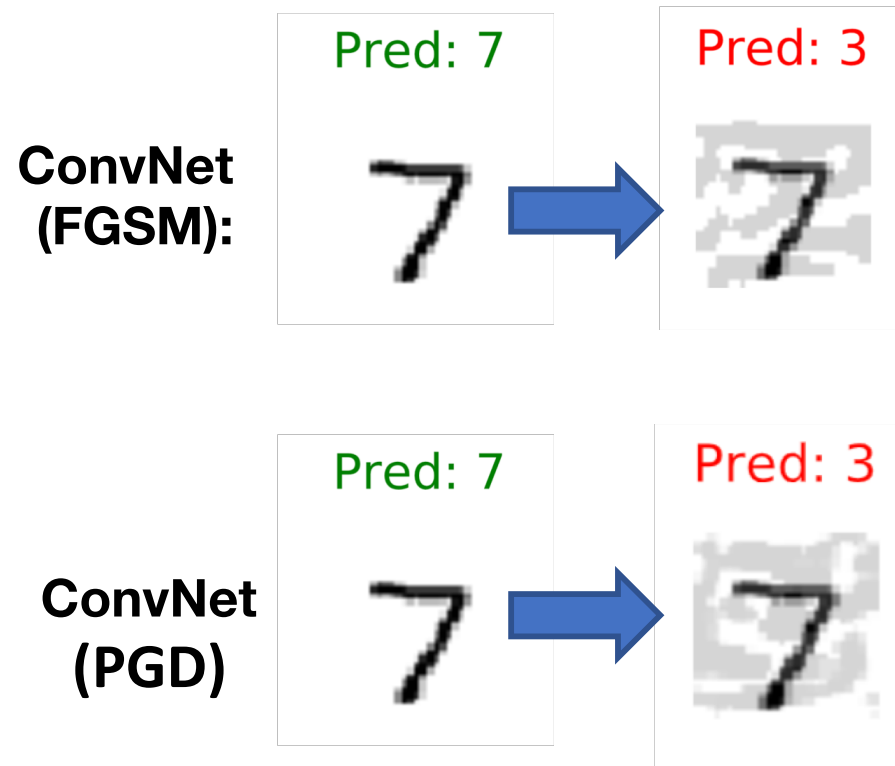
δ_i^* can be found using any attack algorithm such as FGSM, PGD, CW

Note: In practice, one can mix standard updates and adversarial updates

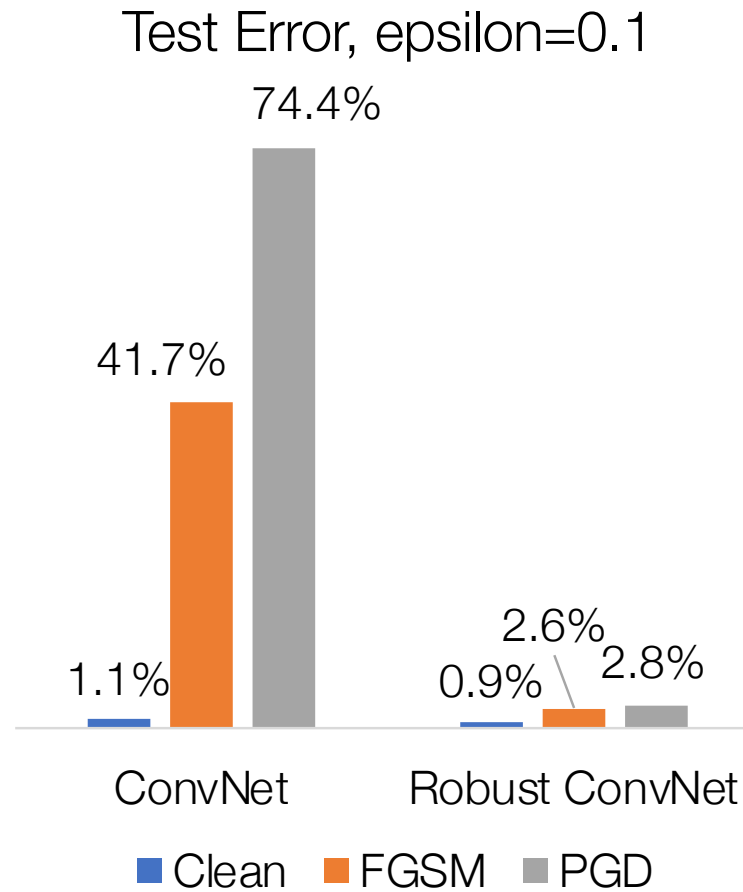
Empirical evaluation



Empirical evaluation



Empirical evaluation



Provably robust training

Inner Maximization

$$\max_{\delta \in \Delta} L(f_{\theta}(x_i + \delta), y_i)$$



Outer Minimization

$$\operatorname{argmin}_{\theta} \sum_{i=1}^n \max_{\delta \in \Delta} L(f_{\theta}(x_i + \delta), y_i)$$

1. Local search via gradients (lower bound on objective)
2. Combinatorial optimization (exact solution on objective)
3. Convex relaxation of the optimization problem (upper bound on objective)

1. Adversarial training
2. N/A
3. Provably robust training